Increment builds

quarta-feira, 6 de agosto de 2025 20:41

Thanks for sharing your script. The issue is now **crystal clear**:
Your current script **deletes the entire build directory every time**, which destroys all cached build information. That's why your build is **never incremental**.

✅ Main Issue

You're doing:
rm -rf ../build
mkdir ../build
cd ../build
This forces a **full rebuild**, because CMake-generated .o and .d files are wiped out. You also re-run cmake .., which is fine — but not necessary unless CMakeLists.txt or project structure changes.

✅ Goal: Preserve build artifacts for incremental builds

Here's how we'll fix it:

- 💡 Keep your build/ directory intact across builds.
- ✅ Re-run cmake .. **only if necessary**.
- 🔄 Add a --full mode if you **really want a clean build**.

✅ Suggested changes — CMakeLists.txt (minor improvements)

In your existing CMakeLists.txt, we'll just clean up and structure better for maintainability and correctness.
New version:

```
cmake_minimum_required(VERSION 3.10)
project(ApiServer)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)  # Useful for clangd and other tools
# Dependencies
find_package(Boost REQUIRED)
find_package(GTest REQUIRED)
find_package(CURL REQUIRED)
# Include directories
include_directories(
  ${Boost_INCLUDE_DIRS}
  ${GTEST_INCLUDE_DIRS}
  include
  /include/Crow/include
  ${CURL_INCLUDE_DIRS}
)
# Source files
file(GLOB_RECURSE SOURCES "${CMAKE_SOURCE_DIR}/src/*.cpp")
file(GLOB_RECURSE HEADERS "${CMAKE_SOURCE_DIR}/src/*.h")
file(GLOB_RECURSE TEST_SOURCES "${CMAKE_SOURCE_DIR}/unitTests/*.cpp")
file(GLOB_RECURSE TEST_SOURCES_TO_TEST
```

```
  "${CMAKE_SOURCE_DIR}/src/Logger/*.cpp"
  "${CMAKE_SOURCE_DIR}/src/ProcessSourceFiles/*.cpp"
  "${CMAKE_SOURCE_DIR}/src/ProcessSourceFiles/util/*.cpp"
  "${CMAKE_SOURCE_DIR}/src/ProcessSourceFiles/Scanner/*.cpp"
  "${CMAKE_SOURCE_DIR}/src/utils/util/*.cpp"
)
# API Server
add_executable(api_server ${SOURCES})
target_link_libraries(api_server PRIVATE Boost::boost CURL::libcurl)
# Unit Tests
add_executable(my_tests ${TEST_SOURCES} ${TEST_SOURCES_TO_TEST})
target_link_libraries(my_tests PRIVATE GTest::gtest GTest::gmock GTest::gtest_main pthread
CURL::libcurl)
target_compile_definitions(my_tests PRIVATE UNIT_TEST)
# Formatter
add_custom_target(
  format
  COMMAND clang-format -i ${SOURCES} ${HEADERS} ${TEST_SOURCES}
  COMMENT "Formatting all C++ source and header files"
)
```

Notes:

- Cleaned formatting for readability.
- CMAKE_EXPORT_COMPILE_COMMANDS is useful for tools like clangd.
- No structural changes—your file layout is fine for now.

✅ Suggested new build.sh script

Here's the **improved version** with incremental builds and a --full option:

```
#!/bin/bash
BUILD_DIR="../build"
if [[ "$1" == "--full" ]]; then
  echo "<> Performing full clean build..."
  rm -rf $BUILD_DIR
  mkdir -p $BUILD_DIR
else
  echo "<> Performing incremental build..."
  mkdir -p $BUILD_DIR
fi
cd $BUILD_DIR || exit 1
# Only rerun CMake if no Makefiles yet
if [ ! -f Makefile ]; then
  echo "<> Running CMake..."
  cmake -DCMAKE_CXX_FLAGS="-fsanitize=address -g" ..
fi
echo "<> Running Auto-format..."
make format
echo "<> Building project..."
make -j$(nproc)
echo "<> Build finished."
echo "<> Running unit tests..."
./my_tests
```

✅ Usage

- Run **incrementally**:
  ./build.sh
- Run **from scratch**:

./build.sh --full

This way, you:

- ✅ Preserve .o and .d files for incremental builds.
- ✅ Reuse cmake config unless structure changes.
- ✅ Maintain flexibility to do a full clean when needed.

🧪 Test to confirm

1. Run ./build.sh
2. Modify *one* .cpp file (e.g., Logger.cpp)
3. Run ./build.sh again — you should see only that .o being rebuilt.

Let me know if you'd like this whole improvement as a PR to your GitHub repo.